

HANDS_ON_file_to_file

Hands-on: Job-Abhängigkeiten ganz ohne Datenbank (File-to-File)

Einsteiger-Track. Keine DBCP/SQLite-Konfiguration nötig: nur Dateien rein, Dateien raus. Ideal als allererster Einstieg, bevor `HANDS_ON_Abhaengigkeiten.md` (mit DB) drankommt. Apache NiFi 2.8.0, UI: <https://localhost:8443/nifi>

Warum file-to-file?

Der einzige Stolperstein der DB-Übungen ist die Controller-Service-Konfiguration (Treiber, URL, Enable-Reihenfolge). Dieser Track zeigt **dieselben Abhängigkeitskonzepte** rein mit `GetFile`, `PutFile`, `UpdateAttribute`, `RouteOnAttribute`, `MergeContent`, `Wait/Notify`: ohne Datenbank.

Pfade

```
Input:      .../nifi-2.8.0/data/input
Output:     .../nifi-2.8.0/data/output
Stage1:     .../nifi-2.8.0/data/output/stage1
Stage2:     .../nifi-2.8.0/data/output/stage2
Archive:    .../nifi-2.8.0/data/archive
Samples:    .../nifi-2.8.0/data/samples
```

Beispieldaten (bereits generiert in `data/samples/`)

`orders.csv`, `customers.csv`, `products.csv`, `sensor_ok.csv`, `orders_broken.csv`

Lab F1: Pipeline / lineare Abhängigkeit (A -> B -> C)

Lernziel: Eine mehrstufige Kette, bei der jede Stufe von der vorigen abhängt: klassische Pipeline.

Flow:

```
GetFile —> UpdateAttribute —> PutFile (stage1) ┐
                                                    └(zweite Kette liest stage1)
```

Einfachste sichtbare Kette: jede Stufe schreibt in einen eigenen Ordner.

Schritte: 1. `GetFile`: Input Directory = Input, Keep Source File = `false`. 2. `UpdateAttribute`: neue Property `verarbeitet_am` = `${now():format("yyyy-MM-dd HH:mm:ss")}` und `stufe` = 1. (Demonstriert: FlowFile-Attribute = Metadaten, die mitlaufen.) 3. `PutFile`: Directory = Stage1. 4. Optional zweite Kette: `GetFile` (Input = Stage1) -> `UpdateAttribute` (`stufe=2`) -> `PutFile` (Stage2).

Verifikation:

```
cp <Samples>/orders.csv <Input>/
ls -l <Stage1>/          # Datei taucht auf -> Stufe 1 fertig
ls -l <Stage2>/          # danach in Stufe 2 -> Kette lief in Reihenfolge
```

Diskussion: Wo “wartet” Stufe 2 auf Stufe 1? (Antwort: in der Eingangs-Queue / im Stage1-Ordner.)

Lab F2: Erfolg vs. Fehler sichtbar machen (success / failure)

Lernziel: Verzweigung nach Relationship: hier mit `RouteOnContent`, um “gültige” von “kaputten” Dateien zu trennen, ganz ohne DB.

Flow:

```
GetFile —> RouteOnContent —match—> PutFile (Output/gut)
                        ↳unmatched—> PutFile (Output/schlecht)
```

Schritte: 1. `GetFile`: Input. 2. `RouteOnContent`: `Match Requirement = content must contain match`; neue Property `gueltig = regulärer Ausdruck ^order_id,datum,menge,betrag` (erwarteter Header). Routing: Treffer -> `gueltig`-Relationship, sonst -> `unmatched`. 3. `gueltig` -> `PutFile Output/gut`; `unmatched` -> `PutFile Output/schlecht`.

Verifikation:

```
cp <Samples>/orders.csv          <Input>/      # korrekter Header -> Output/gut
cp <Samples>/orders_broken.csv   <Input>/      # passt im Header zwar... -> bewusst
diskutieren!
cp <Samples>/sensor_ok.csv       <Input>/      # anderer Header -> Output/schlecht
ls -l <Output>/gut/ <Output>/schlecht/
```

Lehrwert: `orders_broken.csv` hat denselben Header, aber kaputte Zeilen.
`RouteOnContent` prüft nur den Inhalt per Regex, nicht die Struktur: guter Anlass, über “Validierung vs. simple Filter” zu reden.

Lab F3: Fan-in Barriere ohne DB (alle drei zusammenführen)

Lernziel: Synchronisationsbarriere: erst loslegen, wenn alle drei Quelldateien da sind.

Flow:

```
GetFile —> MergeContent —merged—> PutFile (Output/paket)
```

- `MergeContent`: Minimum Number of Entries = 3, Maximum = 3, Max Bin Age = 30 sec, Merge Format = `Binary Concatenation` (oder `Avro/CSV` je nach Wunsch).

Verifikation:

```
cp <Samples>/orders.csv <Input>/
cp <Samples>/customers.csv <Input>/ # noch kein Output
cp <Samples>/products.csv <Input>/ # jetzt erscheint das Paket
ls -l <Output>/paket/
```

Diskussion: Was, wenn nur 2 Dateien kommen? (Max Bin Age greift als Timeout.)

Lab F4: Echte Job-Kopplung mit Wait/Notify (file-to-file)

Lernziel: "Job B erst nach Job A", aber komplett dateibasiert.

Controller Services (einmalig, NiFi 2.x): MapCacheServer + MapCacheClientService (siehe Haupt-Doku). Erst Server, dann Client aktivieren.

Flow:

```
Job A: GetFile (Input) —> PutFile (Archive) —success—> Notify (Signal: dateiX-fertig)
Job B: GenerateFlowFile —> Wait (Signal: dateiX-fertig) —success—> PutFile (Output/freigegeben)
```

Schritte: - Notify: Release Signal Identifier = dateiX-fertig, Cache = Client, success terminieren. - Wait: Release Signal Identifier = dateiX-fertig, Cache = Client, Target Signal Count = 1, success -> PutFile, expired/failure terminieren. - GenerateFlowFile: Run Schedule = 10 sec.

Aha-Effekt: Erst nur Job B starten -> FlowFiles stauen in der Self-Loop-Queue an Wait. Dann Job A starten und Datei einwerfen -> Notify feuert -> Job B schreibt seine Freigabe-Datei.

Praxis-Hinweise (im Live-Test verifiziert): - Die wait-Relationship MUSS als Self-Loop auf Wait zurückgeführt werden (nur expired/failure auto-terminieren). Sonst verschwinden wartende FlowFiles. - Nach dem Notify bis zu ~30s Geduld (Penalty auf der wait-Schleife). - Zwischen Läufen MapCacheServer kurz deaktivieren/aktivieren, um das Signal zu löschen.

```
cp <Samples>/orders.csv <Input>/
ls -l <Archive>/ <Output>/freigegeben/
```

Reset

```
rm -f <Input>/ * <Output>/ **/* <Archive>/ *
# Wait/Notify-Signale: MapCacheServer kurz deaktivieren/aktivieren
```

Mapping zu den DB-Labs

File-to-File	DB-Variante (Haupt-Doku)	Konzept
F1	(Pipeline)	lineare Abhängigkeit A->B->C

File-to-File	DB-Variante (Haupt-Doku)	Konzept
F2	Lab 1	success/failure-Verzweigung
F3	Lab 3	Fan-in-Barriere
F4	Lab 2	Wait/Notify "B nach A"